

CAMPUS QA ASSISTANT PROJECT

校园问答助手开发项目

Day 1 · 前置与环境搭建

技术背景 · 开发环境 · 项目初始化

今日课程导览

01

软件开发行业与项目

行业背景、项目流程、团队分组与角色



02

AI 认知与系统架构

LLM 与 RAG 概览、系统整体架构设计



03

环境搭建与项目初始化

环境 · 脚手架 · *GitLab* · 数据库设计 · 接口设计



01

软件开发行业概览

了解行业背景、项目流程与团队协作



软件开发行业介绍

行业现状

全球软件市场规模持续扩大， AI 驱动新一轮技术变革， 软件工程师需求旺盛

主要技术栈分类

- 前端： *React / Vue / Angular*
- 后端： *Java / Python / Node.js*
- 数据库： *MySQL / MongoDB / Redis*
- AI： *LLM / RAG / 向量数据库*

热门岗位

全栈开发、 AI 应用开发、 *DevOps* 工程师

发展趋势

AI 原生应用、 低代码平台、 云原生架构

关键数据

\$800B+

全球软件市场规模（ 2025 ）

25%+

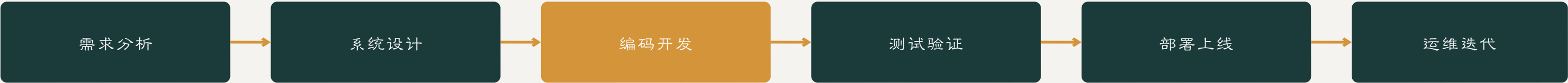
AI 相关岗位年增长率

Top 3

软件工程师薪资排名

项目开发流程详解

标准软件开发生命周期 SDLC



敏捷开发 Scrum 流程

- Sprint 规划** → 明确本次迭代目标与任务分解
- 每日站会** → 15 分钟同步进度与阻塞问题
- 迭代开发** → 持续编码、测试、集成
- 回顾总结** → 复盘改进，进入下一 *Sprint*

本项目 *Sprint* 规划

- 6 天为一个完整 *Sprint* 周期
- 每日站会同步进度
- 关键交付物：代码、文档、演示

关键交付物

- ▶ 需求文档（功能清单、用例描述）
- ▶ 设计文档（数据库设计、接口设计）
- ▶ 源代码（前后端完整代码）
- ▶ 测试报告（功能测试、接口测试）
- ▶ 部署包（可运行的应用包）

实训项目：校园知识问答助手

项目背景

基于大语言模型和 RAG 技术，构建一个支持文档上传、智能问答、历史记录管理的完整 Web 应用

项目目标

实现**用户管理**、**知识库管理**、**AI 问答**、**问答记录**四大核心模块

核心功能模块



用户管理

注册、登录、角色权限管理



知识库管理

文档上传、切分、向量化处理



AI 智能问答

RAG 检索增强、流式回答



问答记录

历史查询、会话管理

技术栈

前端 *React 18 + TypeScript + Vite + Ant Design + Tailwind CSS*

后端 *Spring Boot 3.x + MyBatis-Plus + MySQL 8.0*

AI *OpenAI API / 通义千问 + FAISS 向量库 + Embedding*

工具 *GitLab + Swagger + Postman + VS Code / IntelliJ IDEA*

团队分组与角色分配

项目角色定义



项目经理 *PM*

进度把控与协调，任务分配，
风险管理，团队沟通



前端工程师

用户界面开发，组件实现，
前后端联调，交互优化



后端工程师

API 开发，业务逻辑，数据库
设计，接口联调



AI 算法工程师

RAG 流程实现，*Prompt* 设计，
LLM 接入调优



测试工程师

功能测试，*Bug* 追踪，质量
保障，测试报告

分组规则

分组原则

根据技能特长均衡分配，每组 6 人

建议配置

1 名 *PM* + 1-2 名前端 + 1-2 名后端 + 1 名 *AI* 工程师

小型团队可一人兼任多角色

今日输出

- ✓ 确定分组名单与角色分配表
- ✓ 每组明确各自职责与任务
- ✓ 建立团队沟通机制

02

AI 认知与系统架构

LLM/RAG 概览与系统整体架构



大语言模型（LLM）基础

什么是大语言模型

基于 *Transformer* 架构，通过海量文本训练得到的生成式 *AI* 模型，能够理解和生成自然语言

代表性模型

GPT-4 / *Claude* / 文心一言 / 通义千问 / *DeepSeek*

LLM 核心能力

- 文本生成 — 写文章、邮件、报告等各类文本
- 代码编写 — 生成、解释、调试代码
- 知识问答 — 回答各领域问题
- 逻辑推理 — 数学计算、逻辑分析

LLM 的局限性



知识截止

训练数据有截止日期，不了解最新事件



幻觉 Hallucination

可能生成看似合理但实际错误的内容



无法访问私有数据

不能读取企业内部文档和私有知识



上下文长度限制

一次能处理的文本量有限（4K-128K）

→ 这些局限性正是 *RAG* 技术要解决的问题

RAG 检索增强生成原理

RAG = *Retrieval-Augmented Generation* , 将**信息检索**与**文本生成**相结合的技术框架, 让 *LLM* 能够基于外部知识回答问题

RAG 工作流程



RAG 核心优势



解决知识不足

让 *LLM* 能够访问训练数据之外的知识



支持私有数据

企业可基于内部文档构建专属问答系统



答案可溯源

可追溯到引用文档, 提高可信度



减少幻觉

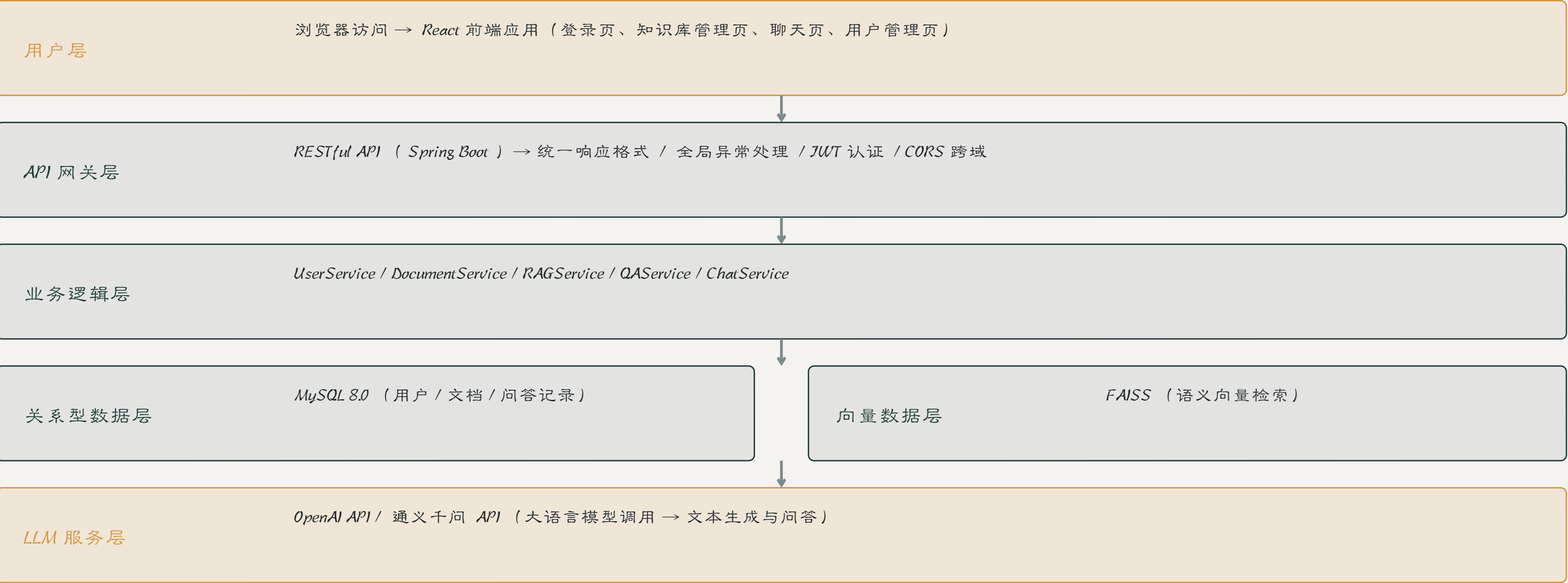
基于检索到的真实内容生成回答

RAG 核心组件

文档切分 → 向量化 (Embedding) → 向量数据库 → 检索排序 → Prompt 工程

系统整体架构设计

系统架构概览



03

环境搭建与项目初始化

开发环境 · 脚手架 · *GitLab* · 数据库 · 接口设计



开发环境搭建

前端环境

Node.js 18+ / npm / yarn

React 18 + TypeScript

Vite 构建工具

后端环境

JDK 17+

Maven / Gradle

Spring Boot 3.x

数据库环境

MySQL 8.0

Redis （可选缓存）

字符集 utf8mb4

AI 环境

Python 3.10+

FAISS 向量库

OpenAI SDK

开发工具

VS Code / IntelliJ IDEA

Git 版本控制

Postman API 测试

验证检查清单

- ✓ 前端 dev 正常启动
- ✓ 后端服务端口监听
- ✓ 数据库连接成功

今日输出

- ✓ 完成开发环境搭建与验证
- ✓ 前后端项目可正常运行
- ✓ 数据库连接配置完成

前后端脚手架搭建

前端脚手架

创建命令

`npm create vite@latest frontend -- --template react-ts`

目录结构

`src/components | src/pages | src/services | src/utils | src/hooks`

配置 `ESLint + Prettier + Tailwind CSS`

后端脚手架

创建方式

`Spring Initializr` (`start.spring.io`)

依赖集成

`Spring Web + MyBatis-Plus + MySQL Driver + Lombok`

目录 `controller | service | mapper | entity | config`

前后端联调配置

CORS 跨域配置： 后端配置允许前端域名访问

统一响应格式： `Result/ResponseVO { code, message, data, timestamp }`

Axios 封装： 请求拦截（加Token）、响应拦截（统一错误处理）

代理配置： `vite.config.ts` 中配置 `/api` 代理到后端服务

代码管理工具 *GitLab* 使用

GitLab 账号与项目

注册个人账号，创建项目代码仓库

Git 基础操作

clone → *add* → *commit* → *push* → *pull* → *branch*

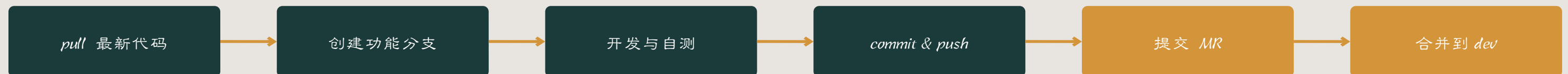
分支管理策略

main（主分支） / *dev*（开发分支） / *feature/**（功能分支）

团队协作规范

- 提交信息规范（*Conventional Commits*）
- 代码审查（*Code Review*）
- 合并请求（*Merge Request*）
- 冲突解决流程
- 每日提交代码习惯

日常工作流程



Demo 程序运行测试

前端 Demo 测试

启动命令： `npm run dev`

验证点： 开发服务器正常启动

页面可在浏览器访问 (`localhost:5173`)

无编译错误，控制台无报错

后端 Demo 测试

启动方式： 运行 `Spring Boot` 主类

验证点： 服务端口监听正常 (`8080`)

控制台无异常日志

`Swagger` 文档可访问

前后端联调测试

- 前端调用后端测试接口
- 验证数据通信正常
- 检查响应格式正确
- 确认 `CORS` 配置生效

常见问题排查

端口冲突 → 修改端口配置

依赖缺失 → 重新安装依赖

CORS 错误 → 检查跨域配置

数据库连接失败 → 检查 `URL` 和密码

数据库环境搭建

MySQL 8.0 安装与配置

字符集设置 `utf8mb4` （支持中文和 `emoji` ）

端口配置（默认 `3306` ）， `root` 密码设置

数据库创建

`CREATE DATABASE campus_qa CHARACTER SET utf8mb4;`

连接池配置

`HikariCP` ：最大连接数 `20` ， 连接超时 `30` 秒

`application.yml` 配置

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/
    campus_qa?useUnicode=true
    username: root
    password: your_password
    driver-class-name:
      com.mysql.cj.jdbc.Driver
```

验证方式

方式一： 使用 `Navicat` / `DBeaver` 等可视化工具连接测试

方式二： 命令行 `mysql -u root -p` 登录验证

方式三： 启动 `Spring Boot` 应用， 观察控制台数据库连接日志

用户表设计

sys_user 表结构

字段名	类型	约束	说明	备注
id	BIGINT	PK, AUTO	主键，自增	
username	VARCHAR(50)	UNIQUE, NOT NULL	用户名	登录账号
password	VARCHAR(100)	NOT NULL	加密密码	BCrypt 加密
email	VARCHAR(100)		邮箱地址	
role	VARCHAR(20)	NOT NULL, DEFAULT	角色	admin / user
status	TINYINT	NOT NULL, DEFAULT	状态	1 启用 0 禁用
create_time	DATETIME	DEFAULT	创建时间	自动填充

设计要点

- 密码使用 BCrypt 加密存储，禁止明文保存
- username 添加唯一索引防止重复注册
- status 字段实现软删除，保留数据可追溯性
- 用户与问答记录为一对多关系

文档表与问答记录表设计

kb_document 文档表

字段名	类型	说明
id	BIGINT PK	主键
title	VARCHAR(200)	文档标题
file_path	VARCHAR(500)	存储路径
file_type	VARCHAR(20)	文件类型 pdf/doc/txt
chunk_count	INT	切分块数
status	VARCHAR(20)	处理状态

qa_record 问答记录表

字段名	类型	说明
id	BIGINT PK	主键
user_id	BIGINT FK	关联用户
question	TEXT	用户问题
answer	LONGTEXT	AI 回答
source_docs	VARCHAR(500)	引用文档 JSON
create_time	DATETIME	创建时间

表关系说明

- sys_user 与 qa_record：一对多关系（一个用户有多条问答记录）
- kb_document 独立管理知识库文档，通过文档标题关联引用来源
- 三张表构成系统的核心数据模型，支撑用户管理、知识库管理和问答记录功能

接口设计 (Swagger)

RESTful API 设计规范

URL 命名： */api/resource/action*

HTTP 方法： *GET* （查） *POST* （增） *PUT* （改） *DELETE* （删）

状态码： 200 成功 201 创建 400 参数错误 401 未授权 500 服务器错误

Swagger 集成

- *SpringDoc OpenAPI* 依赖
- 注解自动生成文档
- 访问 */swagger-ui.html*

用户模块接口示例

接口地址	方法	功能	说明
<i>/api/user/register</i>	<i>POST</i>	用户注册	用户名 / 密码 / 邮箱
<i>/api/user/login</i>	<i>POST</i>	用户登录	返回 <i>Token</i>
<i>/api/user/list</i>	<i>GET</i>	用户列表	支持分页搜索
<i>/api/user/{id}</i>	<i>PUT</i>	更新用户	修改信息
<i>/api/user/{id}</i>	<i>DELETE</i>	删除用户	软删除

Day 1 输出要求

01

人员分组与角色分配

确定分组名单，明确每位成员的角色（PM/前端/后端/AI）

02

系统架构草图

手绘或使用工具绘制系统整体架构图，标注各层组件

03

开发环境搭建

完成前后端及数据库环境搭建，确保项目可正常运行



提交要求

GitLab 提交：将今日完成的代码和文档提交到 *GitLab* 仓库

日报：每位学员提交今日工作日报，记录完成内容和遇到的问题

截止时间：今晚 10:00 前所有输出物提交（课代表统一整理好提交的作业）

Day1 下午 · 环境与初始化输出



建立代码库个人账号

注册 *GitLab* 账号，创建项目仓库，完成初始配置



建立初始化项目

前后端项目脚手架搭建完成，目录结构规范



前后端 *Demo* 测试

前后端程序可正常运行，联调测试通过



前后端用到的控件、组件等清单

列出项目中使用的 *UI* 组件、工具类、依赖库等



数据字典（*SQL* 脚本）

用户表、文档表、问答记录表的完整 *SQL* 建表脚本



提交要求：所有代码和 *SQL* 脚本提交到 *GitLab*，提交信息遵循 *Conventional Commits* 规范

DAY 1 COMPLETED

Day 1 课程小结

了解软件行业与开发流程 → 认识 LLM 与 RAG 技术
→ 明确系统架构 → 完成环境搭建

明天开始正式编码!

Next: Day 2 基础功能开发 — 用户模块与权限控制